React küsib andmed serverilt

11IT · Veebirakendused · 8. tund · konspekt

Selles konspektis selgitan kõik mõisted uuesti läbi, ka need, mida oleme varem kasutanud. See on meelega: mida rohkem kordi sama mõtet kuuled, seda kindlamalt see pähe jääb. Koodi käivitamiseks on vaja Node.js-i, npm-i, VS Code'i ja söökla projekti, mille saad ka kursuse lehelt alla laadida. Lahenda „Proovi ise" kastid, vastused on lõpus.

Pärast seda tundi oskad:
• panna Reacti lehe ja serveri korraga tööle ning selgitada, kes kellega räägib;
• tuua lehe avanemisel andmed serverist ja panna need ekraanile;
• näidata kasutajale laadimisteadet ja veateadet.

Kõige tähtsam on teine punkt. Läbivalt võrdleme kõike restoraniga.

Mis täna muutub?

Meie söökla rakenduse lehel on arvustused. Kust need tulevad? Praegu failist `andmed.ts`, mis asub lehe enda koodi sees. Need on näidisandmed. Justkui toidupilt menüüs: näeb välja nagu päris, aga köögist pole keegi midagi toonud.

Samal ajal on sul olemas ka päris köök ehk server, mis oskab samad arvustused välja anda. Need kaks ei tea teineteisest midagi. Täna ühendame nad. Fail `andmed.ts` läheb prügikasti ja leht küsib andmed serverilt.

Klient, server, päring, vastus

Kõigepealt neli sõna, millel kogu tänane tund seisab.

Restoranis	Veebis	Mis see on
Sina laua taga	**klient**	Programm, mis saadab päringuid. Meil on selleks brauser ja selles töötav Reacti leht.
Tellimus: „üks hernesupp"	**päring**	Küsimus, mille klient saadab.
Köök	**server**	Programm, mis ootab päringuid ja vastab neile.
Taldrik, mis lauale tuuakse	**vastus**	See, mille server tagasi saadab.

Köök ei tee midagi enne, kui keegi tellib. Server samuti: ta käivitub ja jääb ootama. Seepärast ei tule terminali pärast serveri käivitamist uut käsurida. Terminal pole kinni jooksnud, server lihtsalt töötab.

JSON: mis kujul arvustused liiguvad

Interneti kaudu saab saata teksti, pilte, heli ja muid andmeid. Meie rakendus kasutab arvustuste saatmiseks JSON-vormingus teksti. **JSON** (loe „džeison") on kokkulepe, kuidas andmed tekstina kirja panna, et teine pool neist aru saaks. Üks arvustus näeb JSON-ina välja nii:

```
{ "id": 2, "toit": "Hernesupp", "hinne": 3, "kommentaar": "Natuke liiga paks." }
```

Loogelised sulud tähendavad **objekti**: üks asi, millel on nimega väljad. Mitu asja järjest on **massiiv**, selle ümber on nurksulud:

```
[ { "id": 1, "toit": "Kanakaste riisiga" }, { "id": 2, "toit": "Hernesupp" } ]
```

Proovi ise 1. Kas `[ { "id": 1 }, { "id": 2 }, { "id": 3 } ]` on objekt või massiiv? Mitu elementi selles on?

Terminal ja port

Terminal on aken, kuhu kirjutad arvutile käske. Meil on täna neid vaja kaks tükki, sest korraga töötab kaks programmi ja kumbki hõivab oma terminali. Selles projektis on pordid sellised:

Programm	Mida teeb	Käsk	Port
Vite	annab brauserile sinu Reacti lehe	`npm run dev`	5173
Express	annab välja arvustused	`node index.js` kaustas server	3000

Mis on **port**? Mõtle toidusaalile: üks maja, mitu letti, igal letil oma number. Sinu arvuti on maja ja selle aadress on `localhost`, mis tähendab „seesama arvuti“. Port on leti number: selle järgi teatakse, millisele programmile päring läheb. `localhost:3000` tähendab „selles arvutis lett number 3000“. Vite'i vaikimisi port on 5173, aga kui see on juba kinni, võib ta võtta teise. Seepärast ava alati aadress, mille Vite terminali kirjutab.

Kumbki programm jääb tööle, kuni paned ta kinni. Selleks on `Ctrl+C` selles terminalis, kus ta töötab.

Proovi ise 2. Sul on lahti `http://localhost:3000/api/arvustused`. Kumb programm sellele vastab: Vite või Express?

Miks brauser ei lase otse küsida?

Võiks ju arvata, et leht kirjutab lihtsalt `fetch('http://localhost:3000/api/arvustused')` ja kõik. Proovisin. Ei tööta! Brauseri konsoolis on punane veateade sõnaga **CORS**.

Brauser võrdleb lehe ja päringu päritolu: protokolli, hostinime ja porti. Pordid 5173 ja 3000 on erinevad, seega vajab brauser vastuse lugemiseks serveri luba. Meie arendusprojektis kasutame päringu vahendamiseks Vite'i.

Selleks on **proxy** (loe „proksi“) ehk vahendusserver. Restoranis on see teenindaja: sina räägid temaga ja tema viib tellimuse kööki. Brauser küsib Vite'ilt `/api/arvustused`, mis on lehega sama päritoluga. Vite saadab küsimuse serverile edasi ja toob vastuse tagasi.

kumbki server töötab oma terminalis



brauser räägib ainult Vite'iga, Vite räägib serveriga

Päring käib läbi Vite'i, seepärast näeb brauser ainult ühte päritolu.

Pane tähele, et `/api/arvustused` algab kaldkriipsuga ja selles pole serveri nime ega porti. Selline aadress tähendab „sama koht, kust see leht tuli“. Vahendusserveri seadistus läheb faili `vite.config.ts`:

```
server: {
  proxy: {
    '/api': 'http://localhost:3000',
  },
},
```

Inimkeeles: kui aadress algab `/api`-ga, saada päring edasi aadressile `localhost:3000`. Kõik muu annab Vite ise. See seadistus kehtib Vite'i **arendusserveris**. Kui rakendus kunagi päriselt internetti panna, tuleb see lahendada teisiti.

> **Proovi ise 3.** Brauser küsib `/api/menyy`. Kes vastab lõpuks? Aga kui brauser küsib `/favicon.svg`?

React: komponent ja JSX

React on teek, millega ehitatakse veebilehti tükkidest. Üks tükk on **komponent**. Komponent on funktsioon, mille nimi algab suure tähega ja mis tagastab selle, mida ekraanil näidata:

```
export default function App() {
  return <h1>Söökla arvustused</h1>;
}
```

See HTML-i moodi kirjutis on **JSX**. Loogelised sulud JSX-is tähendavad „siin on JavaScript": `{toit}` näitab muutuja `toit` väärtust.

useState: Reacti mälu

Mõtle restorani seinal olevale tahvlile. Kui kirjutad tahvlile uue asja, näevad kõik seda kohe. Reactis on selline tahvel **olek** (ingl *state*) ja see tehakse käsuga `useState`:

```
const [arvustused, setArvustused] = useState<Arvustus[]>([]);
```

Tükk	Tähendus
`arvustused`	mis tahvlil praegu kirjas on
`setArvustused`	funktsioon, millega tahvlile uus asi kirjutada. Pärast seda joonistab React lehe uuesti.
`<Arvustus[]>`	TypeScriptile: siin on massiiv, mille iga element on Arvustus
`([])`	mis on alguses tahvlil: tühi massiiv

Miks mitte tavaline muutuja? Tavalist muutujat muutes React ei tea, et midagi juhtus, ja ekraan jääb samaks.

TypeScript on JavaScript, kuhu saab kirja panna andmete tüübi, näiteks et hinne on arv. Tüüp `Arvustus` asub failis `src/types.ts` ja see jääb alles, kuigi `andmed.ts` kaob.

fetch, async ja await: tellimuse esitamine

Andmeid küsitakse käsuga `fetch`. Päring võtab aega, nagu toit köögist. `fetch()` tagastab kohe Promise'i ehk lubaduse tulevase tulemuse kohta. Nagu teenindaja, kes ütleb „supp on tulemas": supp ise pole veel laual. `await` võimaldab kasutada tulemust pärast seda, kui see on kohale jõudnud.

Meie näites peab `await`-i kasutava funktsiooni ees olema `async`. Funktsioon võib olla `async` ka siis, kui selles pole `await`-i. `await` peatab ainult selle funktsiooni jätkamise, kogu leht ei jää seisma.

```
async function laeArvustused() {
  const vastus = await fetch('/api/arvustused');
  const andmed: Arvustus[] = await vastus.json();
  setArvustused(andmed);
}
```

Kood	Mida see teeb
`async function laeArvustused() {`	Teeme funktsiooni. async lubab selles kasutada sõna await.
`const vastus = await fetch( '/api/arvustused');`	Saadab päringu ja ootab. vastus on nagu kaanega kaetud taldrik: seal on staatus ja sisu, aga kaas on veel peal.
`const andmed: Arvustus[] = await vastus.json();`	Võtab kaane pealt: vastus.json() loeb JSON-vastuse JavaScripti väärtuseks. Ka see võtab aega, sellepärast teine await.
`setArvustused(andmed);`	Kirjutab andmed tahvlile. React joonistab lehe uuesti ja arvustused on ekraanil.

Kirje `: Arvustus[]` aitab TypeScriptil koodi kontrollida, kuid ei kontrolli serveri vastuse tegelikku sisu.

Proovi ise 4. Mis juhtub, kui kirjutad `const vastus = fetch('/api/arvustused');` ilma sõnata await? Kas `vastus` on siis päris vastus?

useEffect: tee see, kui leht avaneb

Funktsioon on olemas, aga keegi pole seda veel välja kutsunud. Millal seda teha? Kohe, kui leht avaneb. Selleks on `useEffect`:

```
useEffect(() => {
  laeArvustused();
}, []);
```

Vaatame seda koodi osade kaupa. `() => { … }` on **noolefunktsioon**, lühidalt kirjutatud funktsioon ilma nimeta. Selle anname Reactile ja ütleme: „käivita see pärast seda, kui leht on ekraanile joonistatud.“

Lõpus olev `[]` on **sõltuvuste massiiv**: nimekiri asjadest, mille muutumisel tuleb funktsioon uuesti käivitada. Tühja sõltuvuste massiiviga `[]` käivitub efekt komponendi lisamisel lehele, mitte iga joonistamise järel.

Miks ei võiks `laeArvustused()` lihtsalt komponenti kirjutada, ilma `useEffect`-ita? Sest iga kord, kui olek muutub, käivitab React kogu komponendi funktsiooni uuesti. Tekiks ring: lae andmed, kirjuta tahvlile, joonista uuesti, lae andmed… Serverile läheks päringuid lõputult.

Ära ehmata: meie projektis on failis `src/main.tsx` sisse lülitatud **StrictMode**, Reacti arendusaegne kontrollrežiim. Selle tõttu toimub arenduse ajal alguses täiendav käivitus. Kui vaatad F12 all vahekaarti Network, näed kahte päringut. See on normaalne.

Mis juhtub, kui midagi läheb valesti?

Server võib olla kinni. Siis peab kasutaja nägema selget teadet, mitte tühja lehte. Vea püüdmiseks on `try` ja `catch`: „proovi seda; kui läks valesti, tee hoopis toda.“

Üks meeldetuletus enne koodi: `true` tähendab „tõene“ ja `false` „väär“. Hüüumärk `!` pöörab selle ümber, `!true` on `false`.

```
  async function laeArvustused() {
    try {
      const vastus = await fetch('/api/arvustused');
      if (!vastus.ok) {
        throw new Error('Server vastas veaga');
      }
      const andmed: Arvustus[] = await vastus.json();
      setArvustused(andmed);
    } catch {
      setViga('Arvustusi ei saanud kätte. Kas server töötab?');
    }
    setLaeb(false);
  }
```

Kood	Mida see teeb
`try { … } catch { … }`	Kui try sees tekib viga, hüppab programm kohe catch-i.
`if (!vastus.ok) {`	vastus.ok on true, kui server vastas õnnestunult (staatus 200–299). !vastus.ok tähendab: kui ei läinud hästi.
`throw new Error('Server vastas veaga');`	Tekitab ise vea, nii et programm hüppab catch-i. fetch ei pea veakoodiga vastust veaks, nii et peame ise kontrollima.
`setViga(...)`	Paneb veateksti olekusse, et seda näidata.
`setLaeb(false);`	Laadimine sai läbi, ükskõik kas õnnestus või mitte. Sellepärast on see rida väljaspool try-d.

Ekraanil näitame teateid nii:

```
{laeb && <p className="kokku">Laen arvustusi…</p>}
{viga && <p className="kokku">{viga}</p>}
```

`&&` tähendab JSX-is: kui vasak pool on tõene, näita paremat poolt. Tühi tekst `''` loetakse vääraks, nii et kui viga pole, ei näidata midagi.

> **Proovi ise 5.** Kui `laeb` on `false` ja `viga` on `''`, mis teade ekraanil on?

Kogu lugu järjest

1. Avad lehe. Vite annab brauserile Reacti koodi.
2. React käivitab App-i. `arvustused` on tühi ja `laeb` on tõene, ekraanil on „Laen arvustusi…“.
3. `useEffect` käivitab `laeArvustused`.
4. `fetch` saadab päringu Vite’ile, Vite saadab selle edasi pordile 3000.
5. Express leiab marsruudi ja vastab JSON-iga. **Marsruut** on reegel „sellele aadressile vasta nii“.
6. `setArvustused` kirjutab andmed tahvlile ja `setLaeb(false)` lõpetab laadimise.
7. React joonistab uuesti. Arvustused on ekraanil.

Kui midagi ei tööta

Mida näed	Võimalik põhjus	Mida kontrollida
„Arvustusi ei saanud kätte…“	Server ei tööta, aadress on vale või vahendusserver pole seadistatud.	Serveri terminal, Vite'i terminal, F12 all Network: mis staatus päringul on. Fail `vite.config.ts`.
Punane CORS-i viga konsoolis	fetch-is on täielik aadress koos pordiga.	Kas fetch-is on ainult `/api/arvustused`.
Konsoolis tuhandeid päringuid	`[]` on puudu või `laeArvustused()` on väljaspool `useEffect`-i.	`useEffect`-i viimane rida.
`map is not a function`	Olekus ei ole massiiv, näiteks puudub `await vastus.json()` või server saatis muud andmed.	Teine `await` ja F12 all Network → Response.
`Cannot find module ...` `andmed`	Fail on kustutatud, aga import on alles.	Kas App-is on veel rida `import { naidisArvustused } ...`.

Sõnastik

Sõna	Tähendus
klient	programm, mis saadab päringuid; meil brauser ja Reacti leht
server	programm, mis ootab päringuid ja vastab neile
päring, vastus	küsimus ja see, mis selle peale tagasi tuleb
marsruut	serveri reegel: sellele aadressile vasta nii
JSON	kokkulepe, kuidas andmeid tekstina kirjutada
port	number, mis ütleb, millisele programmile arvutis päring läheb
localhost	seesama arvuti
päritolu	protokoll, hostinimi ja port kokku
CORS	reeglid, millega server saab lubada brauseril lugeda teise päritoluga vastuseid
proxy, vahendusserver	programm, mis saadab päringud edasi; meil Vite
olek (state)	Reacti mälu; selle muutmisel joonistatakse leht uuesti
useEffect	Reacti käsk: tee see pärast joonistamist
Promise	lubadus tulevase tulemuse kohta
await, async	„kasuta tulemust, kui see on kohal“; ja funktsioon, kus nii tohib teha
try, catch	proovi; kui läks valesti, tee hoopis nii

„Proovi ise“ vastused

1. Massiiv, sest ümber on nurksulud. Selles on kolm elementi.
2. Express, sest port on 3000.
3. `/api/menyy` peale vastab Express, sest aadress algab `/api`-ga. `/favicon.svg` peale vastab Vite ise.
4. Ei. Ilma await-ita on `vastus` alles Promise ehk lubadus, et vastus kunagi tuleb.
5. Ei ühtegi. Mõlemad on väärad.